ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO ĐỒ ÁN MÔN HỌC

MÔN: THỰC HÀNH PHÁT TRIỂN HỆ THỐNG TRÍ TUỆ NHÂN TẠO

Hệ Thống Nhận Diện Khuôn Mặt

Kiến trúc Enterprise Microservices và Điện toán Biên

Nhóm thực hiện: Nhóm Thực Hành Phát Triển TTNT
Thành viên:
Nguyễn Hồng Anh - 24022255
Nguyễn Đức Bình - 24022269
Phạm Gia Bảo - 24022267
Đào Gia Thế Vũ - 24022491

Ngày nộp: 27 tháng 5, 2026

Mục lục

PHẦN MỞ ĐẦU **3**

1 TỔNG QUAN DỰ ÁN VÀ BÀI TOÁN KINH DOANH **3**
1.1 Bối cảnh và Nỗi đau thị trường . . . 3
1.2 Mục tiêu và Phạm vi dự án . . . 3
1.3 Đối chiếu Yêu cầu Đồ án . . . 3

2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG **4**
2.1 Trí tuệ nhân tạo & Thị giác máy tính . . . 4
2.2 Kiến trúc Hệ thống Phân tán (Distributed Systems) . . . 4
2.3 Tổng quan Bộ Công nghệ . . . 4

3 PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC TỔNG THỂ **4**
3.1 Phân tích Use-case . . . 4
3.2 Kiến trúc 9 Node Container . . . 5
3.3 Vai trò của API Gateway (Nginx) . . . 6

4 CHIẾN LƯỢC LƯU TRỮ ĐA TẦNG (POLYGLOT PERSISTENCE) **6**
4.1 PostgreSQL (Source of Truth) . . . 6
4.2 Qdrant (Vector Database) . . . 7
4.3 MinIO (Object Storage) . . . 7

5 PIPELINE AI NHẬN DIỆN KHUÔN MẶT **7**
5.1 Cấu hình AI Engine . . . 7
5.2 Thuật toán Đối sánh . . . 7
5.3 Xử lý Ngoại lệ AI . . . 7

6 BACKEND API VÀ XỬ LÝ BẤT ĐỒNG BỘ **7**
6.1 Thiết kế Endpoint Core . . . 7
6.2 Giải pháp Message Queue . . . 8
6.3 Luồng Xử lý ngầm (Background Worker) . . . 8

7 THIẾT KẾ GIAO DIỆN VÀ ĐIỆN TOÁN BIÊN **8**
7.1 Điện toán Biên (Edge Computing) . . . 8
7.2 User Terminal (Giao diện Kiosk) . . . 8
7.3 Admin Console (Dashboard Quản trị) . . . 9
7.4 Giao diện Quản trị Cơ sở dữ liệu (DB Managers) . . . 9

8 BẢO MẬT, ĐÓNG GÓI VÀ TRIỂN KHAI **10**
8.1 Quản trị Xác thực . . . 10
8.2 Bảo vệ Mã nguồn . . . 10
8.3 Triển khai Docker . . . 11

9 VẬN HÀNH TỰ ĐỘNG VÀ KIỂM THỬ **11**
9.1 Kịch bản Tự động hóa Vận hành . . . 11
9.2 Kịch bản Kiểm thử . . . 11

10 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN **11**
10.1 Phân công Nhóm 11
10.2 Kết quả đạt được & Hạn chế 11
10.3 Hướng phát triển tương lai 12

TÀI LIỆU THAM KHẢO **12**

PHẦN MỞ ĐẦU

Tóm tắt báo cáo:

Báo cáo trình bày toàn bộ vòng đời phát triển của Hệ thống nhận diện khuôn mặt tự động phục vụ chấm công. Điểm khác biệt cốt lõi của dự án là việc áp dụng kiến trúc phân tán (Microservices) chuẩn doanh nghiệp, lưu trữ đa tầng (Polyglot Persistence) với Qdrant, MinIO, PostgreSQL, và xử lý bất đồng bộ qua Redis Queue. Giao diện được tối ưu hóa theo mô hình điện toán biên (Edge Computing) kết hợp định tuyến an toàn qua Nginx Reverse Proxy.

1 TỔNG QUAN DỰ ÁN VÀ BÀI TOÁN KINH DOANH

1.1 Bối cảnh và Nỗi đau thị trường

Trong kỷ nguyên chuyển đổi số, việc quản lý nhân sự và chấm công tại các doanh nghiệp, trường học đang đòi hỏi sự tự động hóa cao. Các phương pháp truyền thống như thẻ từ, vân tay tồn tại nhiều "nỗi đau"(pain points):

- Thẻ từ: Dễ quên, dễ mất, dễ xảy ra tình trạng "chấm công hộ".
- Vân tay: Không đảm bảo vệ sinh (đặc biệt sau đại dịch), thiết bị dễ hỏng do mồ hôi, vân tay bị mờ.

Từ đó, nhu cầu về một hệ thống định danh "Không chạm"(Contactless) bằng khuôn mặt trở thành xu hướng tất yếu.

1.2 Mục tiêu và Phạm vi dự án

Dự án hướng tới xây dựng một MVP (Minimum Viable Product) hoàn chỉnh cho hệ thống điểm danh bằng khuôn mặt thời gian thực. Phạm vi: Triển khai luồng đăng ký khuôn mặt, nhận diện chấm công trực tiếp trên trình duyệt (Kiosk) và hệ thống quản trị nội bộ (Admin Dashboard). Tiêu chí: Tốc độ nhận diện dưới 2 giây, kiến trúc chịu tải tốt, dễ dàng mở rộng quy mô (Scale-up).

1.3 Đối chiếu Yêu cầu Đồ án

- Yêu cầu cơ bản (Đã đạt): Tích hợp thành công model AI phát hiện và nhận diện khuôn mặt, lưu trữ được lịch sử ra/vào.
- Điểm sáng tự nghiên cứu (Advanced): Không dùng code nguyên khối (Monolith) mà triển khai Microservices bằng Docker. Tích hợp cơ sở dữ liệu Vector (Qdrant), Object Storage (MinIO), Message Broker (Redis) và API Gateway (Nginx).

2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ ÁP DỤNG

2.1 Trí tuệ nhân tạo & Thị giác máy tính

Hệ thống sử dụng phương pháp Face Embeddings. Thay vì so sánh ảnh pixel-by-pixel, mô hình AI (ArcFace) sẽ phát hiện khuôn mặt bằng thuật toán SSD, sau đó trích xuất các đặc trưng sinh trắc học thành một không gian vector đa chiều (cụ thể là mảng số thực 512 chiều). Việc so sánh hai khuôn mặt được quy về bài toán tính khoảng cách hình học giữa hai vector.

2.2 Kiến trúc Hệ thống Phân tán (Distributed Systems)

- Microservices: Phân rã ứng dụng thành các dịch vụ nhỏ (AI, UI, DB, Queue) chạy trong các Container độc lập. Rủi ro ở dịch vụ này không làm sập toàn bộ hệ thống.
- Message Queue: Áp dụng mô hình Publisher/Subscriber qua Redis. Các tác vụ AI trích xuất vector nặng nề được đẩy vào hàng đợi để Worker xử lý ngầm, giúp API phản hồi tức thì cho người dùng.

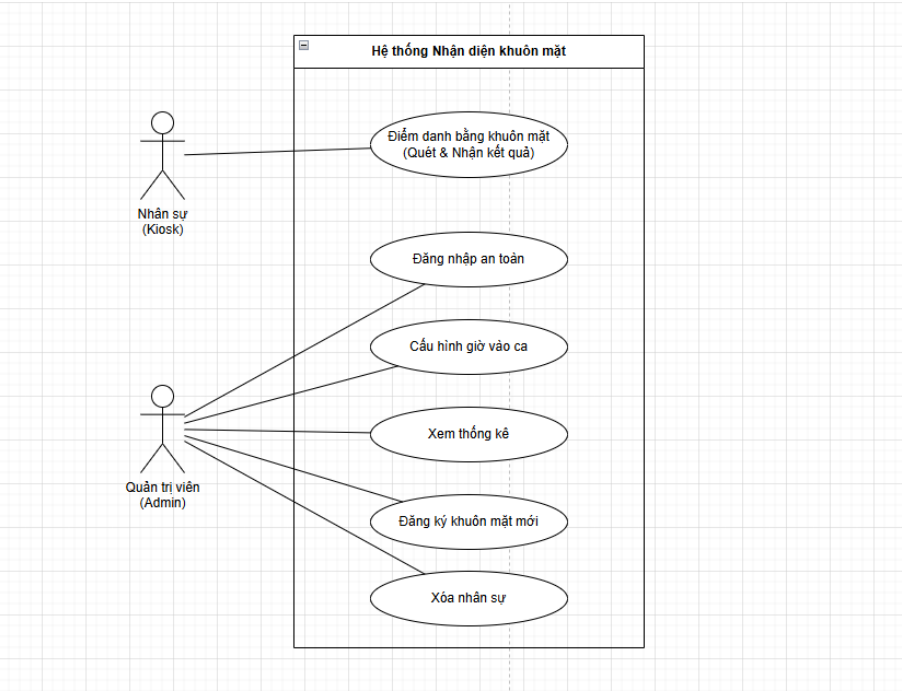
2.3 Tổng quan Bộ Công nghệ

- Backend: Python 3.10, FastAPI (hiệu năng cao, hỗ trợ ASGI).
- Frontend: ReactJS 18, Vite, TailwindCSS.
- DevOps: Docker, Docker Compose, Nginx (Reverse Proxy).

3 PHÂN TÍCH YÊU CẦU VÀ KIẾN TRÚC TỔNG THỂ

3.1 Phân tích Use-case

- Actor - Nhân sự (Kiosk): Đứng trước Camera, hệ thống tự động quét, hiển thị Bounding Box (khung vẽ khuôn mặt) và thông báo kết quả (Đúng giờ/Đi muộn/Người lạ).
- Actor - Quản trị viên (Admin): Đăng nhập an toàn, cấu hình giờ vào ca, xem thống kê, đăng ký khuôn mặt mới hoặc xóa vĩnh viễn nhân sự khỏi cơ sở dữ liệu.

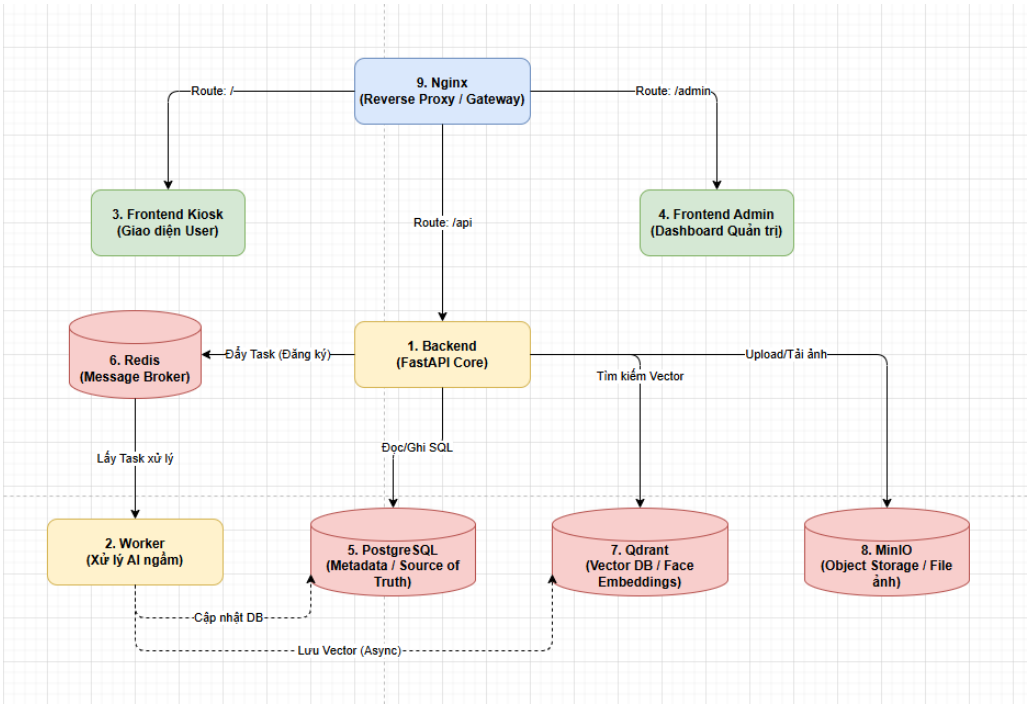


Hình 1: Sơ đồ Use-case của hệ thống

3.2 Kiến trúc 9 Node Container

Dự án được cấu hình bằng `docker-compose.yml` gồm 9 Node độc lập:

1. `backend`: FastAPI xử lý logic cốt lõi.
2. `worker`: Xử lý hàng đợi AI ngầm.
3. `frontend_kiosk`: Giao diện người dùng.
4. `frontend_admin`: Giao diện quản trị.
5. `postgres`: DB quan hệ.
6. `redis`: Message Broker.
7. `qdrant`: Vector DB.
8. `minio`: Object Storage.
9. `nginx`: Reverse Proxy/Load Balancer.



Hình 2: Sơ đồ kiến trúc luồng thực thi 9 Node Container

3.3 Vai trò của API Gateway (Nginx)

Đây là nút thắt kiến trúc cực kỳ quan trọng. Để giải quyết lỗi CORS (Cross-Origin Resource Sharing) giữa các port khác nhau và tăng tính bảo mật, tất cả các port nội bộ (8000, 3000, 4000) đều được ẩn đi. Nginx mở duy nhất cổng 80 và định tuyến theo sub-path:

- `/` → Frontend Kiosk
- `/admin/` → Frontend Admin
- `/api/` → Backend FastAPI

4 CHIẾN LƯỢC LƯU TRỮ ĐA TẦNG (POLYGLOT PERSISTENCE)

Hệ thống loại bỏ tư duy lưu nhồi nhét mọi thứ vào SQL bằng cách chia nhỏ thành 3 tầng:

4.1 PostgreSQL (Source of Truth)

Đóng vai trò lưu trữ Meta-data (dữ liệu có cấu trúc). Bao gồm các bảng `users` (tài khoản admin), `face_profiles` (ánh xạ mã sinh viên với link ảnh) và `attendance` (lịch sử quét mặt).

4.2 Qdrant (Vector Database)

Lưu trữ đặc trưng khuôn mặt dưới dạng Vector 512 chiều. Qdrant được tối ưu để tính toán đại số tuyến tính siêu tốc. Collection `face_embeddings` được cấu hình sử dụng thuật toán tính khoảng cách Cosine (Cosine Distance).

4.3 MinIO (Object Storage)

Đóng vai trò như AWS S3 thu nhỏ. Các ảnh gốc (.jpg) upload lên hệ thống không bị nén vào blob của Postgres mà được lưu file tĩnh tại MinIO bucket `face-images`. Giảm thiểu phình to CSDL quan hệ.

5 PIPELINE AI NHẬN DIỆN KHUÔN MẶT

5.1 Cấu hình AI Engine

Sử dụng thư viện DeepFace. Quá trình khởi động ứng dụng (Event startup của FastAPI) được cấu hình một hàm "Warm-up"để nạp model ArcFace vào RAM/VRAM trước, tránh độ trễ ở lần quét đầu tiên.

5.2 Thuật toán Đối sánh

Sử dụng hàm `qdrant_client.query_points`. Khi một vector mới đi vào, Qdrant tìm kiếm vector gần nhất. Ngưỡng chấp nhận (Threshold) được thiết lập tại `score > 0.45` dựa trên độ đo Cosine. Nếu vượt qua, xác nhận là người quen.

5.3 Xử lý Ngoại lệ AI

Hệ thống xử lý chặt chẽ các trường hợp rác đầu vào (Noisy Data):

- Ảnh không có người: AI trả về lỗi ValueError, API bọc trong Try-Except để trả về thông báo lỗi thân thiện.
- Ảnh có $>$ 1 người trong khung hình: Hệ thống trả về cờ warning, cảnh báo "Phát hiện nhiều khuôn mặt"nhằm đảm bảo tính toàn vẹn của kết quả điểm danh.

6 BACKEND API VÀ XỬ LÝ BẤT ĐỒNG BỘ

6.1 Thiết kế Endpoint Core

Hệ thống tuân thủ kiến trúc RESTful. Các API tiêu biểu:

- `POST /api/extract`: Trích xuất vector test.
- `POST /api/recognize`: Điểm danh (Chụp ảnh $\rightarrow$ Trích xuất $\rightarrow$ Search Qdrant $\rightarrow$ Ghi Postgres $\rightarrow$ Tính trễ giờ).
- `DELETE /api/users/{code}`: Xóa chuỗi đồng bộ trên 3 DB (Postgres, MinIO, Qdrant).

6.2 Giải pháp Message Queue

Tích hợp thư viện `rq` (Redis Queue). Redis đóng vai trò In-memory Broker điều phối luồng tin nhắn giữa Backend và Worker.

6.3 Luồng Xử lý ngầm (Background Worker)

Tại API Đăng ký (`/api/register`), nếu hệ thống bắt người dùng chờ AI trích xuất và lưu vào Qdrant sẽ gây nghẽn kết nối HTTP. Giải pháp: API chỉ upload ảnh lên MinIO, sau đó đẩy tín hiệu `task_queue.enqueue` vào Redis. Trả về kết quả "Đăng ký thành công"ngay lập tức. Một Container `worker` chạy ngầm hàm `process_face_recognition` sẽ từ từ lấy ảnh xuống và phân tích AI.

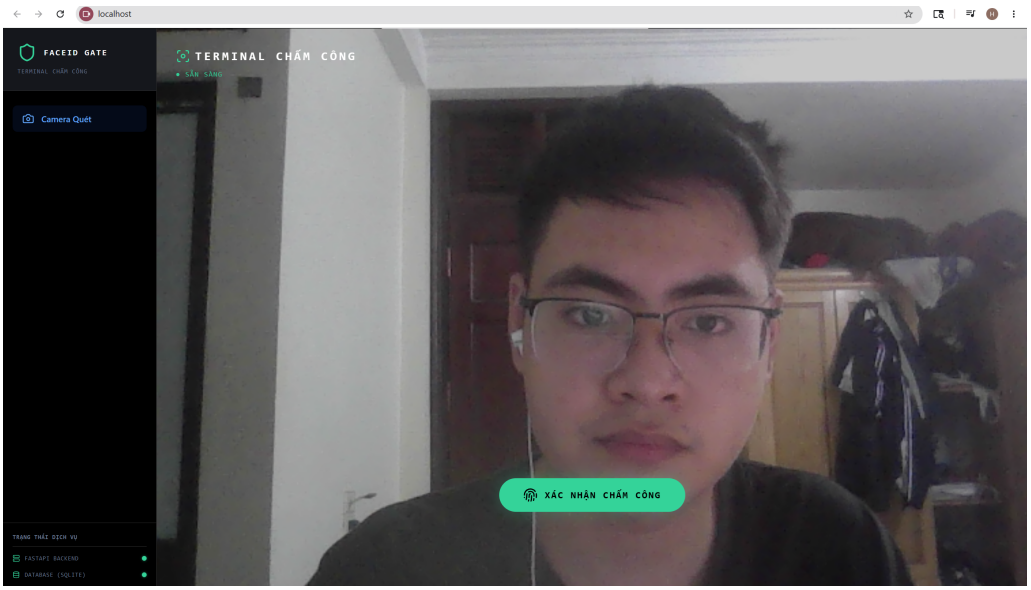
7 THIẾT KẾ GIAO DIỆN VÀ ĐIỆN TOÁN BIÊN

7.1 Điện toán Biên (Edge Computing)

Thay vì truyền luồng video liên tục (Streaming) bằng WebSockets/WebRTC về server gây tốn băng thông cực lớn, nhóm đánh đổi kiến trúc: Xử lý video ngay tại Client (Trình duyệt). Khi bấm "Xác nhận", Client tự cắt một khung hình (`canvas.toBlob`), nén thành JPEG và gọi duy nhất 1 luồng REST API.

7.2 User Terminal (Giao diện Kiosk)

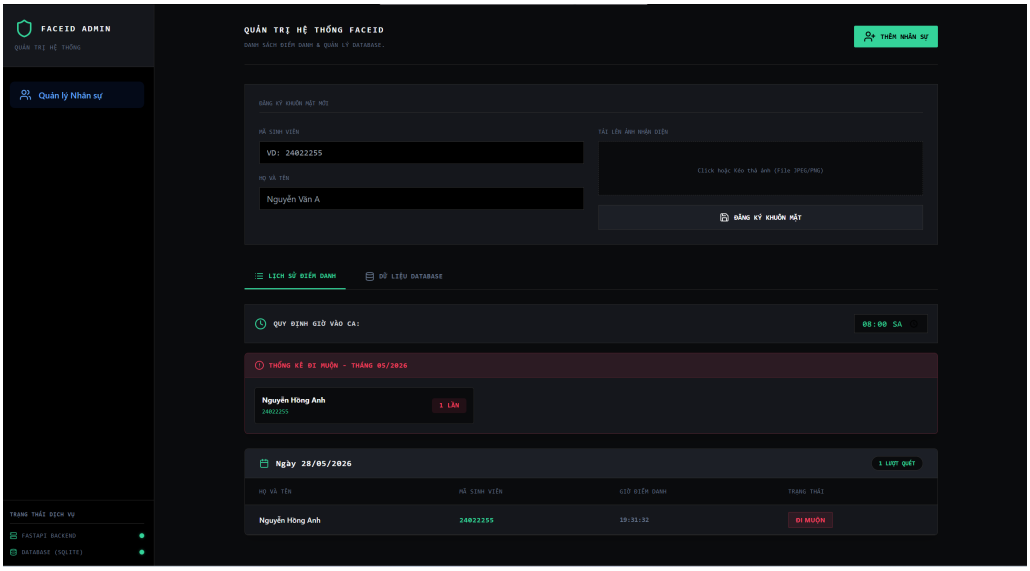
Sử dụng `navigator.mediaDevices.getUserMedia`. Khi nhận được response từ API chứa biến `box` (x,y,w,h), Frontend sử dụng Canvas API 2D để vẽ Bounding Box đè lên luồng Video (Màu xanh: Verified, Màu đỏ: Unknown).



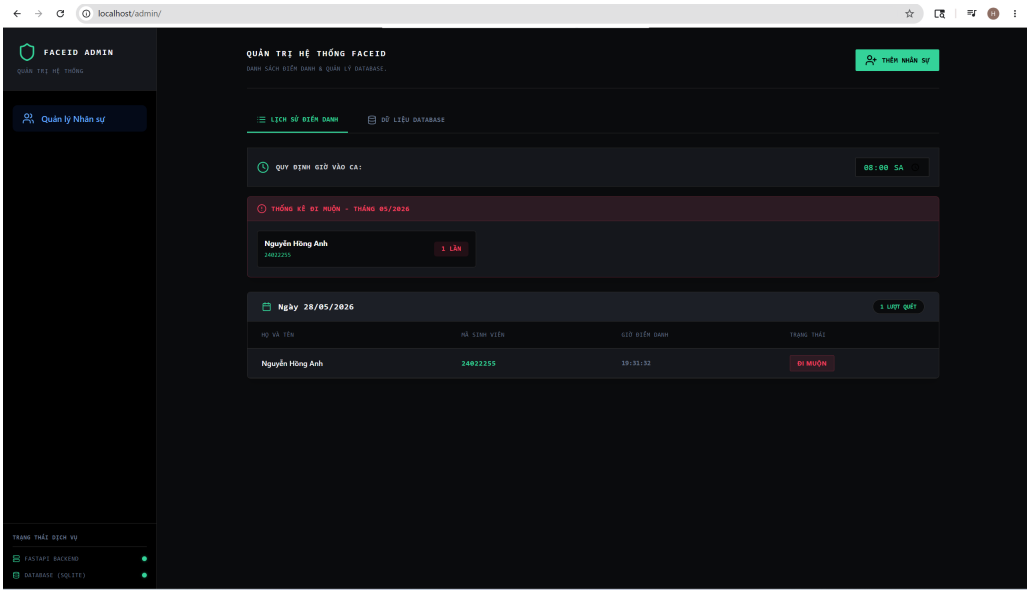
Hình 3: Giao diện Kiosk điểm danh thực tế với Bounding Box tracking khuôn mặt

7.3 Admin Console (Dashboard Quản trị)

Thiết kế SPA: Ứng dụng Single Page Application với các Tabs chuyển đổi mượt mà. Chức năng: Cho phép cấu hình "Giờ vào ca"lưu trực tiếp xuống Database. Hiển thị danh sách log điểm danh. Thống kê số lần đi muộn của từng cá nhân theo tháng trực quan. Quản lý (xem ảnh/xóa) Database sinh viên.



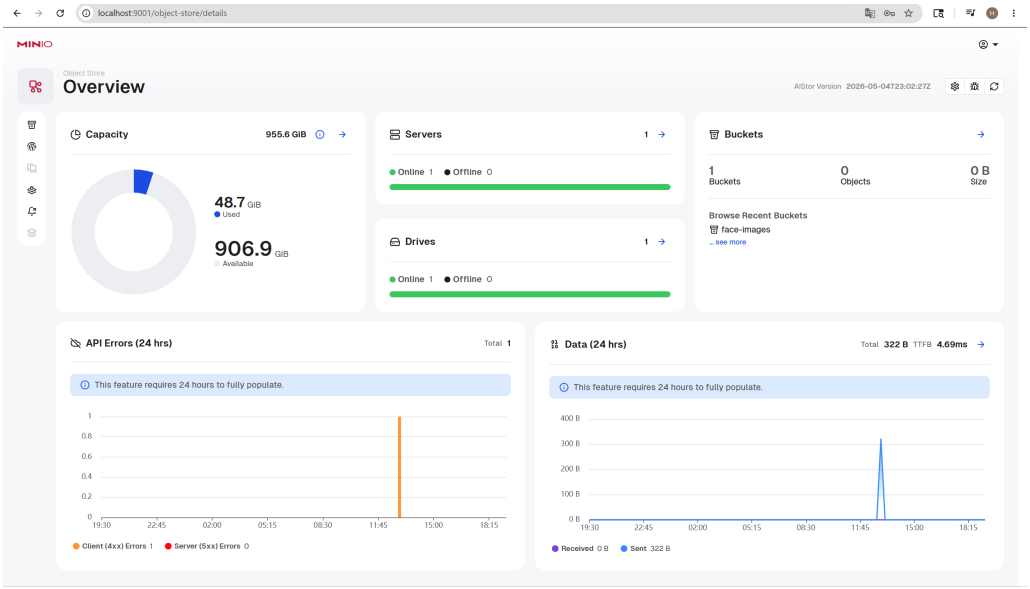
Hình 4: Giao diện Quản trị viên - Form đăng ký nhân sự bằng AI



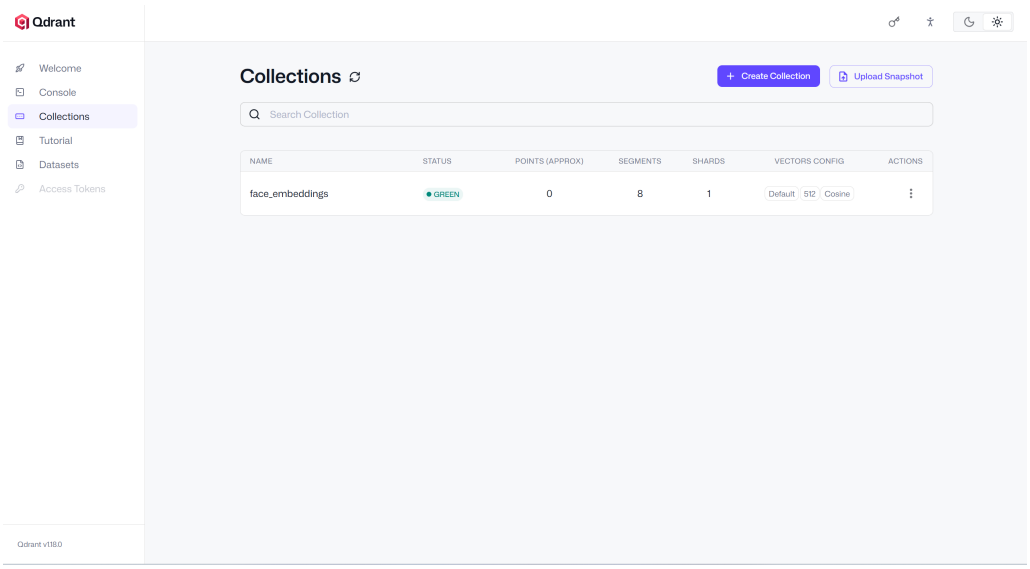
Hình 5: Giao diện Quản trị viên - Thống kê lịch sử điểm danh

7.4 Giao diện Quản trị Cơ sở dữ liệu (DB Managers)

Để trực quan hóa luồng dữ liệu đa tầng, hệ thống tích hợp các giao diện quản trị cơ sở dữ liệu nội bộ.



Hình 6: Giao diện MinIO Console quản lý ảnh gốc (Object Storage)



Hình 7: Giao diện Qdrant Dashboard trực quan hóa không gian Vector 512 chiều

8 BẢO MẬT, ĐÓNG GÓI VÀ TRIỂN KHAI

8.1 Quản trị Xác thực

Trang Admin và các API quan trọng được bảo vệ bằng cơ chế JWT (JSON Web Token). Thuật toán mã hóa HS256 với thời gian sống của Token là 60 phút.

8.2 Bảo vệ Mã nguồn

Toàn bộ mật khẩu, thông tin kết nối DB (`SECRET_KEY`, `POSTGRES_PASSWORD`, `MINIO_SECRET_KEY`) không hardcode trong mã nguồn mà được truyền qua file `.env` và Docker Environment Variables.

8.3 Triển khai Docker

- Backend: Dockerfile tối ưu kích thước bằng `python:3.10-slim`, cài đặt thư viện lõi hệ thống (`libgl1`) cho OpenCV.
- Frontend: Sử dụng Multi-stage build (Dùng Node.js build ra HTML/CSS/JS tĩnh, sau đó copy sang image `nginx:alpine` để serve), giúp image cuối cùng cực kỳ nhẹ.

9 VẬN HÀNH TỰ ĐỘNG VÀ KIỂM THỬ

9.1 Kịch bản Tự động hóa Vận hành

- Data Retention: Để CSDL không bị phình to vô hạn, hàm `log_attendance` có nhúng logic tự động quét và xóa (`db.delete()`) các bản ghi lịch sử điểm danh đã cũ hơn 45 ngày.
- Data Backup: Viết kịch bản Python (`backup_script.py`) chạy lệnh `pg_dump` đối với PostgreSQL và gọi API tạo snapshot đối với Qdrant. Giúp tạo điểm khôi phục an toàn (Disaster Recovery).

9.2 Kịch bản Kiểm thử

Hệ thống phản hồi thao tác quét mặt ổn định dưới mức 2.5 giây. Tính năng Nginx Proxy hoạt động hoàn hảo, che giấu hoàn toàn hệ thống backend nội bộ, giải quyết dứt điểm lỗi CORS Policy trên trình duyệt.

10 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

10.1 Phân công Nhóm

1. Nguyễn Hồng Anh (MSV: 24022255): Phụ trách Core AI (DeepFace, ArcFace) và xây dựng RESTful API Backend bằng FastAPI.
2. Nguyễn Đức Bình: Phụ trách kiến trúc CSDL đa tầng (Postgres, MinIO, Qdrant) và tích hợp Redis Message Queue.
3. Phạm Gia Bảo (MSV: 24022267): Phát triển UI/UX Frontend bằng ReactJS, xử lý luồng Video, vẽ Bounding Box và tích hợp API.
4. Đào Gia Thế Vũ (MSV: 24022491): Phụ trách DevOps, viết Docker Compose, cấu hình Nginx Reverse Proxy điều hướng mạng, và biên soạn Báo cáo Tài liệu kỹ thuật.

10.2 Kết quả đạt được & Hạn chế

Đạt được: Xây dựng thành công MVP ứng dụng AI vào thực tiễn với kiến trúc phần mềm hiện đại, quy chuẩn cấp doanh nghiệp.

Hạn chế: Hệ thống hiện tại chỉ trích xuất hình ảnh 2D, có thể bị qua mặt bởi hình ảnh in trên giấy hoặc video phát trên điện thoại.

10.3 Hướng phát triển tương lai

- Liveness Detection: Tích hợp mô hình AI phụ để phát hiện chớp mắt hoặc chiều sâu khuôn mặt, chống giả mạo (Anti-spoofing).
- IoT & RTSP Camera: Thay vì dùng Webcam trình duyệt, sẽ kết nối trực tiếp luồng stream RTSP từ Camera IP an ninh (Hikvision, Dahua).
- Cloud Kubernetes: Nâng cấp từ Docker Compose lên cụm K8s (Kubernetes) để hệ thống tự động Scale-up số lượng Container AI khi có hàng nghìn công nhân cùng quét mặt vào lúc 8h sáng.

TÀI LIỆU THAM KHẢO

1. Tài liệu chính thức của FastAPI (https://fastapi.tiangolo.com).
2. Tài liệu mô hình DeepFace (https://github.com/serengil/deepface).
3. Tài liệu Qdrant Vector Database (https://qdrant.tech).
4. Tài liệu React Router và Vite.
5. Kiến trúc Microservices (https://microservices.io).